Original Paper

MEASUREMENT AND CONTROL

Measurement and Control
2021, Vol. 54(3-4) 417–428

DOI: 10.1177/00202940211000380
journals.sagepub.com/home/mac
SAGE

A real-time HIL control system on rotary inverted pendulum hardware platform based on double deep Q-network

Yanyan Dai, KiDong Lee and SukGyu Lee

Abstract

For real applications, rotary inverted pendulum systems have been known as the basic model in nonlinear control systems. If researchers have no deep understanding of control, it is difficult to control a rotary inverted pendulum platform using classic control engineering models, as shown in section 2.1. Therefore, without classic control theory, this paper controls the platform by training and testing reinforcement learning algorithm. Many recent achievements in reinforcement learning (RL) have become possible, but there is a lack of research to quickly test high-frequency RL algorithms using real hardware environment. In this paper, we propose a real-time Hardware-in-the-loop (HIL) control system to train and test the deep reinforcement learning algorithm from simulation to real hardware implementation. The Double Deep Q-Network (DDQN) with prioritized experience replay reinforcement learning algorithm, without a deep understanding of classical control engineering, is used to implement the agent. For the real experiment, to swing up the rotary inverted pendulum and make the pendulum smoothly move, we define 21 actions to swing up and balance the pendulum. Comparing Deep Q-Network (DQN), the DDQN with prioritized experience replay algorithm removes the overestimate of Q value and decreases the training time. Finally, this paper shows the experiment results with comparisons of classic control theory and different reinforcement learning algorithms.

Keywords

Double deep Q-network with prioritized experience replay, reinforcement learning, real-time HIL control system, rotary inverted pendulum platform

Date received: 1 February 2021; accepted: 16 February 2021

Introduction

Along with analysis and control from linear systems to nonlinear systems, designing the control system becomes more and more sophisticated. Linearization is an approximation and it is not considered as a good cornerstone to develop a global control law.[1] Analysis of nonlinear systems contains more complicated mathematics.[2,3] The dynamics of a nonlinear system are richer. Therefore, the process of formulating a control law through a standard design process needs deeper understanding and more sophistication.[4] The control of pendulum models has been chosen as a challenging testing ground for nonlinear dynamical models and control theory.[5] As an important branch of automatic control technology, the inverted pendulum system is a typical balance control and an example of an underdriven nonlinear control system. The inverted pendulum system has a serious degree of nonlinearity, high-order instability, and also contains many variables. The inverted pendulum is not only an important experimental device, but also an important applied device. Therefore, the in-depth study of the inverted pendulum system has great theoretical value and urgent display significance. Inverted pendulum systems[6] have been known as the basic model for real engineering applications. Rocket launching and Missile guidance are applied to the behavior of the inverted pendulum.[7,8] A self-balancing unicycle is similar to a two-dimensional inverted pendulum with a unicycle cart at its base.[5] Commercial application for the inverted pendulum model is the Segway,[9] consisting of a pendulum attached to a base platform that has a wheel at each

Yeungnam University, Gyeongsan, South Korea

Corresponding author:
KiDong Lee, Department of Robotics and Intelligent Machine Engineering, Yeungnam University, Gyeongsangbuk-do, Gyeongsan, South Korea.
Email: kdrhee@yu.ac.kr

side. Robotic limb behavior is like a controlled inverted pendulum.[10]

Deep learning has made a huge contribution to the scalability and performance of machines.[11] The Sequential decision-making setting of reinforcement learning and control is an interesting application.[12] Reinforcement learning[13] is concerned with good learning control policies for sequential decision problems, by optimizing a cumulative future reward signal. Q-learning[14] is one of the most popular reinforcement learning algorithms. However, it is well known to learn unrealistically high action values, since it includes a maximization step that exceeds the estimated action value, which tends to overestimate rather than underestimated values.[15] The problem with overestimation is that the agent always selects non-optimal operations in any given state because it has the largest Q value. Overestimations cause insufficiently flexible function approximation[16] and noise.[17] The double Q-Learning algorithm[17] was first proposed in a tabular setting, which can be generalized to work for solving the problem of overestimations of action value in basic Q-Learning. Two different action-value functions Q and Q' are used as estimators in the Double Q-learning algorithm. Although Q and Q' are noisy, these noises can be regarded as uniformly distributed. Therefore, this algorithm solves the overestimation problem. Double Deep Q-Network (DDQN) is proposed in van Hasselt et al.[18] which is to implement Double Q-Learning with Deep Neural Network. Deep Q Network and a target Network are used in the DDQN algorithm. In this paper, we use the DDQN algorithm to obtain more accurate estimation values. In the DQN algorithm, in order to break the relationship between the samples, the experience memory is used to randomly extract the experience update parameters. However, in the case of sparse rewards, only after N multiple correct actions, there is a reward. There will be very few samples that can motivate the agent to learn correctly. The random sampling experience method will be very inefficient, and many samples will be rewarded. To solve this problem, two methods are considered: experience storage method and experience extraction method. At present, the method of experience extraction is mainly used. Priority experience replay is to extract the most important experience first when extracting the experience, but you can't only extract the most important experience, otherwise, it will cause overfitting. It should be the more important the experience, the greater the probability of extraction. In Schaul et al.[19] a framework for prioritizing experience is developed, in order to replay important transitions more frequently and learn more efficiently.

Due to nonlinear feature and complex internal dynamics, it is challenged to design a controller for rotary inverted pendulum using classic control theory. Therefore, without classic control theory, this paper controls the platform by training and testing reinforcement learning algorithm. Reinforcement learning is to model how human beings learn. People try to act on the current state of the environment and obtain rewards. After a few trials, people begin to predict the next state they will, based on the current state and preferred actions. All this information has been strengthened, and in a given state, people know what actions will be taken to maximize their immediate and future rewards, because they know the final result. For the particular rotary inverted pendulum, actions are turning left and turning right of the arm. Environment is the simulation. States are angle of the pendulum, angle of the arm, angular velocity of the pendulum, and angular velocity of the arm. The reward is calculated based on the angle of the pendulum and angle of the arm. When the pendulum gets upright and the arm is in the central position, the reward will be zero. With the help of training and testing tools such as OpenAI Gym and Deepmind Control Suite, many recent successes in reinforcement learning (RL) have become possible. Unfortunately, there is still a lack of research or tools for quickly testing high-frequency RL algorithms and transferring them from simulation to real hardware environments.[20] In Polzounov and Redden[20] a control tool is used to train and test reinforcement learning algorithms on a rotary inverted pendulum platform. However, in this paper, the swing-up control process is based on the nonlinear control system. If researchers have no deep understanding of control, it is difficult to control a rotary inverted pendulum platform using classic control engineering models. In order to successfully control the platform, the settings of initial states and constants used in the control equations of these models are particularly important.[21] In Kim et al.[21] it controls rotary inverted pendulum using deep reinforcement learning rather than classical control engineering. However, in Kim et al.[21] the actual reinforcement learning algorithm attempts to balance the pendulum upright, but not including swing up the pendulum. This paper researches on swing up and balancing the pendulum using reinforcement learning algorithm.

The inverted pendulum system can be viewed abstractly as a control problem with the center of gravity at the top and the mass point at the bottom. Without the interference of external forces, the inverted pendulum system can easily and quickly occur complex and unpredictable changes. Therefore, the control system needs to have the ability to respond to and solve the rapid and unpredictable changes of the inverted pendulum. Hardware-in-the-loop (HIL) is a technique that is used in the development and test of complex real-time embedded system. By using HIL, development time and cost can be significantly reduced. When developing electrical machinery components or systems, the use of computer simulation and actual experiments have been independent of each other. However, by using the HIL approach, these two processes can be combined and show a great improvement in efficiency. In this paper, we create a real-time Hardware-in-the-loop (HIL) control system to swing up and balance the

pendulum using a deep reinforcement learning algorithm rather than classical control engineering. The control system includes four parts: rotary inverted pendulum platform part, HIL interface software part, RL environment part, and agent part. In the HIL interface part, the real-time control software is used to read/write all the input and output channels on the data acquisition (DAQ) device, as well as the system's actuators and sensors. RL environment part receives the rotary inverted pendulum state and sends an action to the rotary inverted pendulum by TCP/IP communication. The Double Deep Q-Network (DDQN) with prioritized experience replay reinforcement learning algorithm is proposed to implement the agent. For the real experiment, in order to swing up the rotary inverted pendulum and make the pendulum smoothly move, the action should be continuous. Twenty-one actions are used to swing up and balance the pendulum. The voltage to control the DC motor is constrained in the range of [−10, 10] Volt.

The article is organized as follows. The second section describes real-time HIL control system architecture. The third section describes how to use double deep Q-network (DDQN) with prioritized experience replay reinforcement learning algorithm to swing up and balance the real rotary inverted pendulum. Finally, the fourth section presents the simulation and comparison of experimental results.

Real-time HIL control system architecture

Rotary inverted pendulum

The Quanser rotary inverted pendulum,[22] as Figure 1, consists of a flat arm with a pivot at one end and a metal shaft on the other end. The pivot-end is mounted on top of the rotary servo base unit, which consists of a DC motor in a solid aluminum frame. This DC motor drives the smaller pinion gear through an internal gearbox. The pinion gear is fixed to a larger middle gear that rotates on the load shaft. The position of the load shaft is measured by a high-resolution optical encoder. The encoder is also used to estimate the velocity of the motor. The actual pendulum link is fastened onto the metal shaft. The pendulum is equipped with an encoder, which can digitally measure the pendulum angle. The pendulum is free to rotate 360°. As shown in Figure 2, the arm and pendulum have the length of r and L, respectively. The measurement results of the platform are as:

α: angle of the pendulum. The range of the angle is [−180, 180] degree. When it is completely vertical in the vertical position, the inverted pendulum angle is zero; when it is rotated counter-clockwise, the inverted pendulum angle is positive.

θ: angle of the arm. The range of the angle is constrained in [−90, 90] degree. When the arm is in the central position, the angle is equal to zero. The angle increases positively when it rotates counter-clockwise. The arm should turn in the counter-clockwise direction when the control voltage is positive.

$\dot{\alpha}$: angular velocity of the pendulum.

$\dot{\theta}$: angular velocity of the arm.

Figure 1. Quanser rotary inverted pendulum.

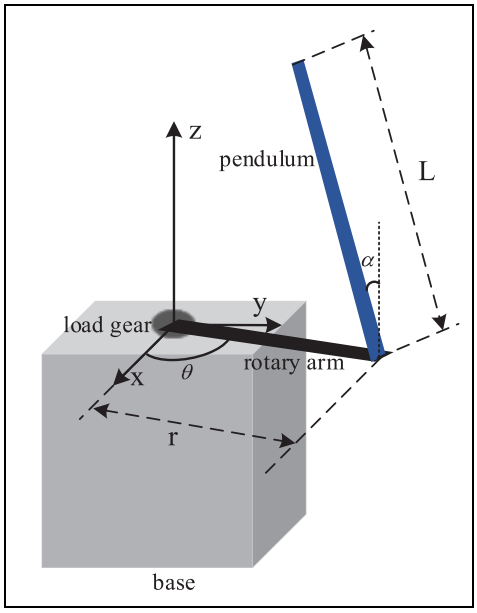

Figure 2. Rotary inverted pendulum conventions.

Based on the courseware shown in Quanser Rotary Inverted Pendulum[22] the Lagrange method is used to

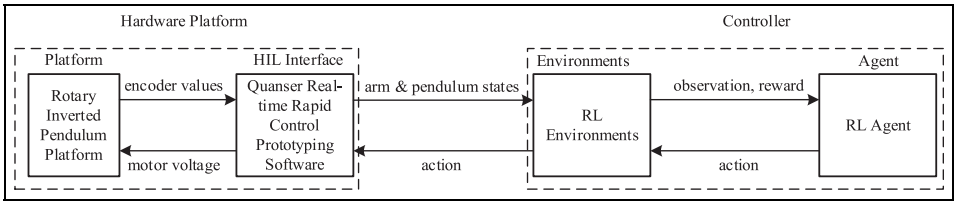

Figure 3. Real-time HIL reinforcement learning system architecture.

obtain the motion equations of the system. With respect to the servo motor voltage, the motions of the rotary arm and the pendulum will described using the Euler-Lagrange equation:

$$\frac{\partial^2 L}{\partial t \partial \dot{q}_i} - \frac{\partial L}{\partial q_i} = Q_i \tag{1}$$

The variables q_i are generalized coordinates. Let

$$q(t)^T = [\,\theta(t) \quad \alpha(t)\,] \tag{2}$$

where, as Figure 2, $\theta(t)$ is the angle of the arm and $\alpha(t)$ is the angle of the pendulum. The velocities are:

$$\dot{q}(t)^T = \left[\frac{\partial\theta(t)}{\partial t} \quad \frac{\partial\alpha(t)}{\partial t}\right] \tag{3}$$

Based on (2) and (3), the Euler-Lagrange equations for the rotary inverted pendulum system are

$$\frac{\partial^2 L}{\partial t \partial \dot{\theta}} - \frac{\partial L}{\partial \theta} = Q_1 \tag{4}$$

$$\frac{\partial^2 L}{\partial t \partial \dot{\alpha}} - \frac{\partial L}{\partial \alpha} = Q_2 \tag{5}$$

The Lagrangian of the system is presented as (6), which is the difference between kinetic of a system and potential energies.

$$L = T - V \tag{6}$$

where T is the total kinetic energy of the system and V is the total potential energy of the system. The generalized forces Q_i are used to describe the nonconservative forces. The generalized force acting on the arm and acting on the pendulum are as (7) and (8), respectively.

$$Q_1 = \tau - B_r\dot{\theta} \tag{7}$$

$$Q_2 = -B_p\dot{\alpha} \tag{8}$$

where B_r is the viscous friction torque of the arm and B_p is the viscous damping coefficient of the pendulum. The Euler-Lagrange equations is a systematic method of finding the equations of motion of a system. The nonlinear equations of motion for the rotary inverted pendulum are:

$$\left(m_p r^2 + \frac{1}{4}m_p L^2(1-\cos(\alpha)^2) + J_r\right)\ddot{\theta} - \left(\frac{1}{2}m_p Lr\cos(\alpha)\right)\ddot{\alpha} + \left(\frac{1}{2}m_p L^2\sin(\alpha)\cos(\alpha)\right)\dot{\theta}\dot{\alpha} + \left(\frac{1}{2}m_p Lr\sin(\alpha)\right)\dot{\alpha}^2 = \tau - B_r\dot{\theta} \tag{9}$$

$$-\frac{1}{2}m_p Lr\cos(\alpha)\ddot{\theta} + \left(J_p + \frac{1}{4}m_p L^2\right)\ddot{\alpha} - \frac{1}{4}m_p L^2\sin(\alpha)\cos(\alpha)\dot{\theta}^2 - \frac{1}{2}m_p Lg\sin(\alpha)) = -B_p\dot{\alpha} \tag{10}$$

where m_p is the mass of pendulum. J_p is the pendulum moment of intertia about center of mass. J_r is the rotary arm moment of inertia about pivot. The torque applied at the base of the rotary arm, which is generated by the servo motor, is calculated as:

$$\tau = \frac{\eta_g K_g \eta_m k_t (V_m - K_g k_m \dot{\theta})}{R_m} \tag{11}$$

where k_t is torque constant. k_m is motor back-emf constant. R_m is terminal resistance. In the experiment, $\eta_g = 0.69$, $K_g = 70$, and $\eta_m = 0.9$.

Real-time HIL reinforcement learning control system

If researchers have no deep understanding of control, it is difficult to control a rotary inverted pendulum platform using classic control engineering models, as shown in section 2.1. Therefore, without classic control theory, this paper controls the platform by training and testing reinforcement learning algorithm. In order to train and test the deep reinforcement learning algorithm using a hardware platform, we create a real-time Hardware-in-the-loop (HIL) reinforcement learning control system. As shown in Figure 3, it consists of four components: rotary inverted pendulum, HIL interface, RL environments, and Agent. The hardware platform includes rotary inverted pendulum and HIL interface. RL environments and agent are processed by controller.

1. Hardware: Rotary inverted pendulum platform. The rotary inverted pendulum is introduced in section 2.1. Except this, the Q8-USB data

acquisition device and VoltPAQ-X1 Amplifier are included. The Quanser Q8-USB is a single-point I/O, eight-channel data acquisition device that delivers reliable real-time performance via a USB interface. The VoltPAQ-X1 amplifier is designed to achieve high performance with Hardware-In-The-Loop (HIL) implementations.
2. HIL Interface. Real-time rapid control prototyping software is used for LabVIEW. The core functions for hardware interface are HIL Initialize, HIL Read, and HIL Write. These three functions enable to read/write to all the input and output channels on the data acquisition (DAQ) device, as well as the system's actuators and sensors. HIL Initialize function configures the Q8 USB DAQ device. HIL Read function is set up to read the position of the load gear using its encoder. The motor voltage is applied to the servo DC motor of the rotary inverted pendulum using DAQ's analog output through the connected power amplifier.
3. RL environments. OpenAI Gym and Tensorflow are used in this step. The core functions of the rotary inverted pendulum environment are the reset, step, render, and close methods. The process gets started by calling reset, which returns an initial observation. Render redraws a frame of the environment, such as popping up a window. When a cycle comes to end, the agent uses this function. The primary element is the step function. The step function takes a time step in the environment based on action and returns observations, rewards, done, and information. The action is chosen by DDQN with prioritized experience replay algorithm based on observations. The action is sent to HIL Interface to control the real rotary inverted pendulum platform by TCP/IP communication. The observations parameters are the angle of the pendulum, angle of the arm, the angular velocity of the pendulum, and angular velocity of the arm, as discussed in section 2.1. The observation parameters are obtained from the HIL interface by TCP/IP communication.
4. Agent. The Double Deep Q-Network (DDQN) with prioritized experience replay reinforcement learning algorithm is used to implement the agent.

Double deep Q-network (DDQN) with prioritized experience replay reinforcement learning algorithm for rotary inverted pendulum

DQN cannot solve the Q value overestimation problem. Overestimation means that the estimated value function is larger than the real value function. If the overestimation is uniform in all states, the action with the largest value function can still be found, according to the greedy strategy. However, the overestimation is not uniform in each state, so overestimation will affect the strategic decision. Therefore, the strategy decision cannot always be obtained. Overestimation is caused by the max operation used in the parameter update or iteration process of the value function. Although using max can quickly make the Q value close to the possible optimization goal, it easily causes the overestimate problem. Overestimation means that the final obtained algorithm model has a big bias. DDQN, like DQN, has the same two Q network structures. Based on DQN, the problem of overestimation is eliminated by decoupling the two steps of target Q action selection and target Q calculation. For DDQN with prioritized experience replay, the batch sampling is not random, but sampling depending on the priority in memory. This will help to effectively find the learning samples. Based on Schaul et al.[19] the SumTree algorithm is used to extract the sample. RL environment part receives the rotary inverted pendulum state and sends an action to the rotary inverted pendulum by TCP/IP communication. The Double Deep Q-Network (DDQN) with prioritized experience replay reinforcement learning algorithm is used to implement the agent. For the real experiment, in order to swing up the rotary inverted pendulum and make the pendulum smoothly move, the action should be continuous. 21 actions are used to swing up and balance the pendulum. The voltage to control the DC motor is constrained in the range of $[-10, 10]$ Volt.

DDQN with prioritized experience replay algorithm update

For the DDQN algorithm update, in each episode, initial observation firstly. The observation values include the angle of the arm, the angle of the pendulum, the angular velocity of the arm, and the angular velocity of the pendulum. Then it will go into the loop, and the steps are processed as below:

1. Put observation into the neural network, and then choose the action, which has the max Q value. Based on DDQN, the output actions are zero and one, which are not continuous. For the real experiment, if the arm position changes largely in a short time, the movement will not be smooth, which will cause the experiment to fails. In order to swing up the rotary inverted pendulum, the action should be continuous. In the experiment, the voltage to control the DC motor is constrained in the range of $[-10, 10]$ Volt. In the experiment, the e-greedy is set as 0.9. In order to make the pendulum swing up, the pendulum should swing clockwise (CW) and counter-clockwise (CCW), so the arm (DC

motor) should turn CW and CCW. Therefore, we define there are 21 actions to swing up the pendulum. We will discrete the output action into 21 actions in the range [−5, 5] Volt, due to safe consideration. 10 actions are used to make the arm turns CCW, and 10 actions are used to make the arm turns CW. The rest one action is used to make the arm in the central position. Based on the knowledge that the pendulum should swing CW and CCW, the action is randomly selected and sent to the environment. The 21 actions are calculated as (1), where $action_{total}$ is equal to 21. The $action_{choose}$ is chosen from array [0,1,2,3,…,18,19,20], based on rotary inverted pendulum states.

$$action = 0.5 * (action_{chooose} - (action_{total} - 1)/2)/((action_{total} - 1)/10) \tag{12}$$

In order to make the pendulum smoothly swing up and protect the hardware, the value of $action_{choose}$ is selected as (13), where $action_{choose}(cur)$ is current $action_{choose}$ value and $action_{choose}(next)$ is next $action_{choose}$ value. $random(0, or1)$ means randomly choosing number 0 or 1. "±" means randomly add or minus the following number.

$$action_{choose}(next) = \begin{cases} action_{choose}(cur) + random(0, \text{or}1) & action_{choose}(cur) = 0 \\ action_{choose}(cur) - random(0, \text{or}1) & action_{choose}(cur) = 20 \\ action_{choose}(cur) \pm random(0, \text{or}1) & action_{choose}(cur) \neq 0, 20 \end{cases} \tag{13}$$

When the pendulum angle is smaller than ±10°, the pendulum will process balance control. The 21 actions should be calculated as (14), where $action_{total}$ is equal to 21. The $action_{choose}$ is chosen from array [0,1,2,3,…,18,19,20], based on rotary inverted pendulum states. For choosing $action_{choose}$, it is the same as (2).

$$action = 0.05 * (action_{chooose} - (action_{total} - 1)/2)/((action_{total} - 1)/10) \tag{14}$$

2. Based on the obtained action, the outputs of the environment are observation, reward, done, and information. The reward is calculated as (15), where α is the angle of the pendulum and θ is the angle of the arm. α_{target} and θ_{target} are the pendulum angle and arm angles at which to success the episode, respectively. They are positive constants. The reward value is the constraint in the range of [−1, 0], in order to efficiently learn. When the pendulum gets upright and the arm is in the central position, the reward will be zero.

$$reward = -\frac{1}{2}\left(\frac{|\alpha - \alpha_{target}|}{\pi} + \frac{2 * |\theta - \theta_{target}|}{\pi}\right) \tag{15}$$

3. The memory stores the current observation, action, reward, and next observation (s, a, r, s'). The memory size is set as 3000. For prioritized experiment replay, in this step, the absolute of TD-error (Temporal Difference error) is calculated and stored.
4. In this step, there are two neural networks: Q_{eval} and Q_{target}. In the learning step, the sampling is extracted based on the absolute of TD-error (TD-error = value of Q_{target}− value of Q_{eval}). If the TD-error is large, it means that the prediction accuracy still has large improvement space, so the sample needs to be learned more and the priority is high. After training, the new absolute of TD-error is updated to the memory. The network of Q_{eval} is used to evaluate the max Q value of the action in the network Q_{target}. The target Q value to train the evaluation network can be described as:

$$Y_{DDQN} = reward' + \gamma Q_{target}(s', argmax Q_{eval}(s', a; \sigma), \sigma') \tag{16}$$

where $reward'$ is the target reward value, and $\gamma = 0.9$ is the reward decay value. σ and σ' are the network parameters. Based on replace target iteration value, every replace target iteration steps, $\sigma' = \sigma$.

DDQN neural network

The architecture of the Deep Q-network is shown in Figure 4. The inputs of the neural network are the rotary inverted pendulum states, such as, arm angle, pendulum angle, arm angular velocity, and pendulum angular velocity. The second layer has 20 neurons. The third layer has 20 neurons. These two layers use the ReLU activation function. The output layer has 21 neurons. The output layer output the Q value of actions. We choose the action, which contains the max Q value. In order to do the experiment, we discrete the action into 21 actions, as discussed in section 3.1. Then one action is taken and send to the environment. The learning rate is 0.005.

Experiment

This section presents experiment results of the proposed real-time HIL control system. The Double Deep

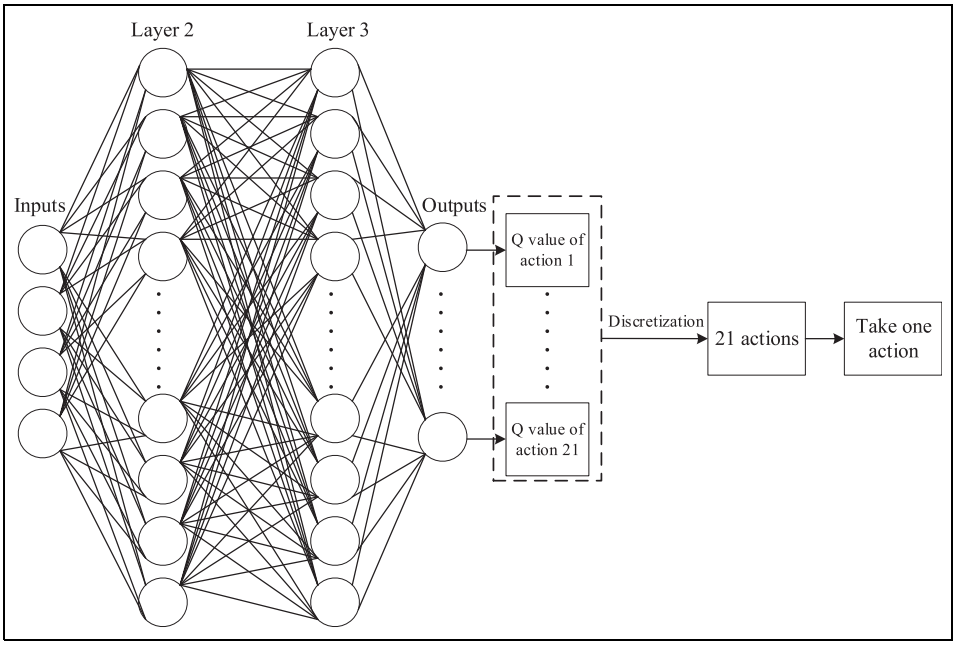

Figure 4. The architecture of the Deep Q network. The input layer was the observation of the rotary inverted pendulum state, the output layer is Q-values for each action.

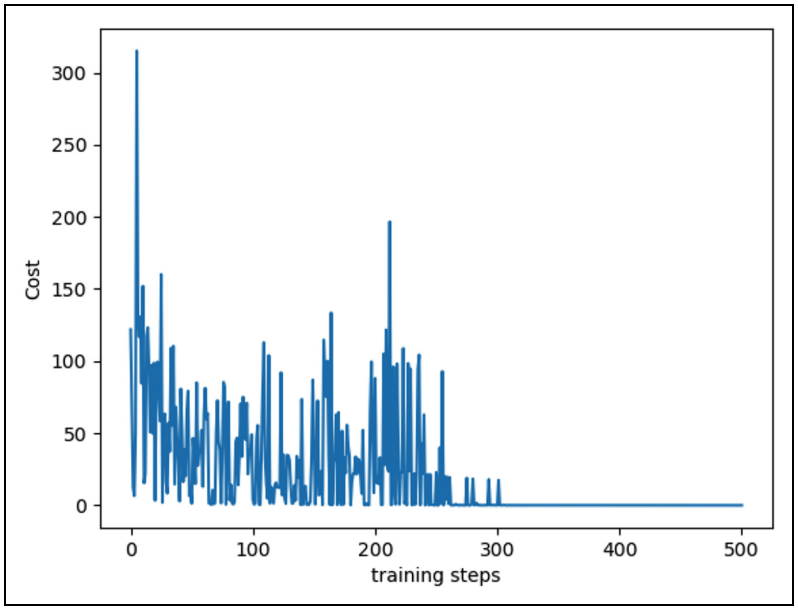

Figure 5. The training steps and cost curve.

Q-Network (DDQN) with prioritized experience replay reinforcement learning algorithm is used to implement the agent. The learning rate is 0.005, reward decay is 0.9, e-greedy is 0.9. Replace target iteration is 200, which determines steps that the target network changes network parameters. The memory size is 3000. The batch size is 32, which is the number of data extracted from the memory every time. Figure 5 shows the learning curve. The cost function is calculated based on the output Q value of network Q_{eval} and output Q value of network Q_{target}. At the first 300 training steps, the rotary inverted pendulum is in the swing-up process, and then it is on the balancing process. Reinforcement learning does not have all the data at the beginning, it collects data through continuous exploration. The previous data may not have a good effect. By continuously expanding the memory size or continuously collecting updated data, due to new data, the cost may suddenly increase. Therefore, in the continuous learning process of reinforcement learning, the cost curve is an

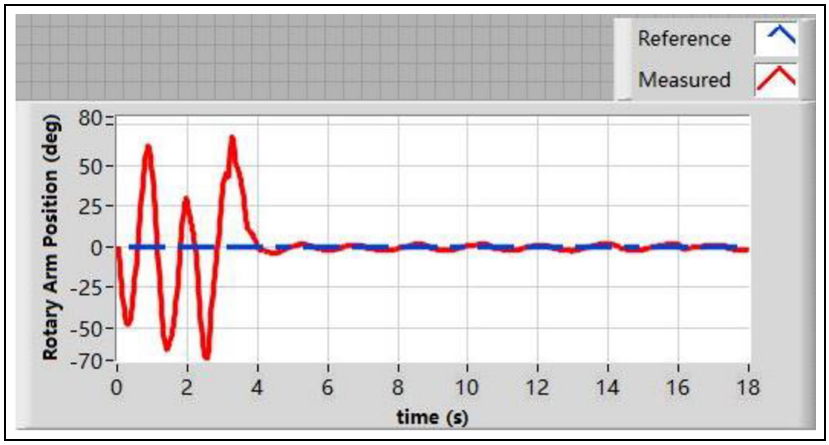

Figure 6. Rotary arm angle based on DDQN with prioritized experience replay algorithm.

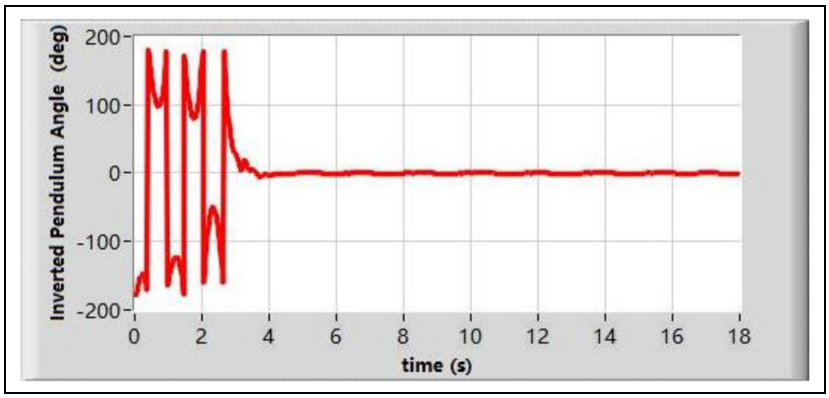

Figure 7. Inverted pendulum angle based on DDQN with prioritized experience replay algorithm.

oscillation curve. When the rotary inverted pendulum is on the balancing process, the pendulum is in a stable state, so the cost is reduced much and stable.

Based on the DDQN with prioritized experience replay algorithm, the rotary inverted pendulum can swing up and balance the pendulum efficiently and effectively. The target angles of the rotary arm and the inverted pendulum are set as 0°. Figures 6 and 7 show the rotary arm angle and the inverted pendulum angle in the experiment. At the first 0–4 s, the rotary inverted pendulum is on the swing up process. Then in the balancing state, the rotary arm and the inverted pendulum are on the target angle state. Figure 8 shows the rendered image of the environment. The rotary inverted pendulum keeps balance after swinging up the pendulum. Figure 9 presents snapshots of rotary inverted pendulum swing-up and balancing experiment using DDQN with prioritized experience replay. (a) at $t = 0$ s, the pendulum is in the downright position; (b) at $t = 2$ s the pendulum is on the swing-up process; (c) at $t = 5$ s the pendulum is in the upright position, and the rotary arm is in the central position; (d) at $t = 11$ s, the pendulum is on the balancing process. The experiment video is uploaded to drop box. The link is in Appendix 1.

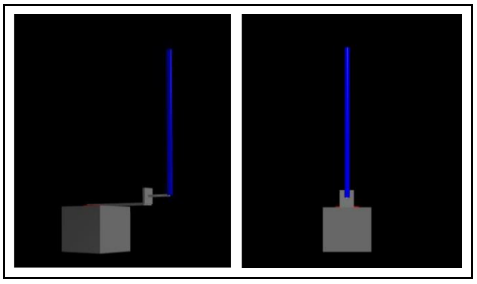

Figure 8. Rendered image of the environment. The rotary inverted pendulum keeps balance after swinging up the pendulum.

In order to compare with our results in Figures 6 and 7, the rotary inverted pendulum is processed based on the classic control theory. Energy-based control theory is used to swing the pendulum up from its downward position. During balancing process, pole placement is used to design the controller. Figures 10 and 11 present the rotary arm angle and the inverted pendulum angle in the experiment, respectively. The experiment video is uploaded to dropbox. The link is shown in Appendix 2. The rotary inverted pendulum

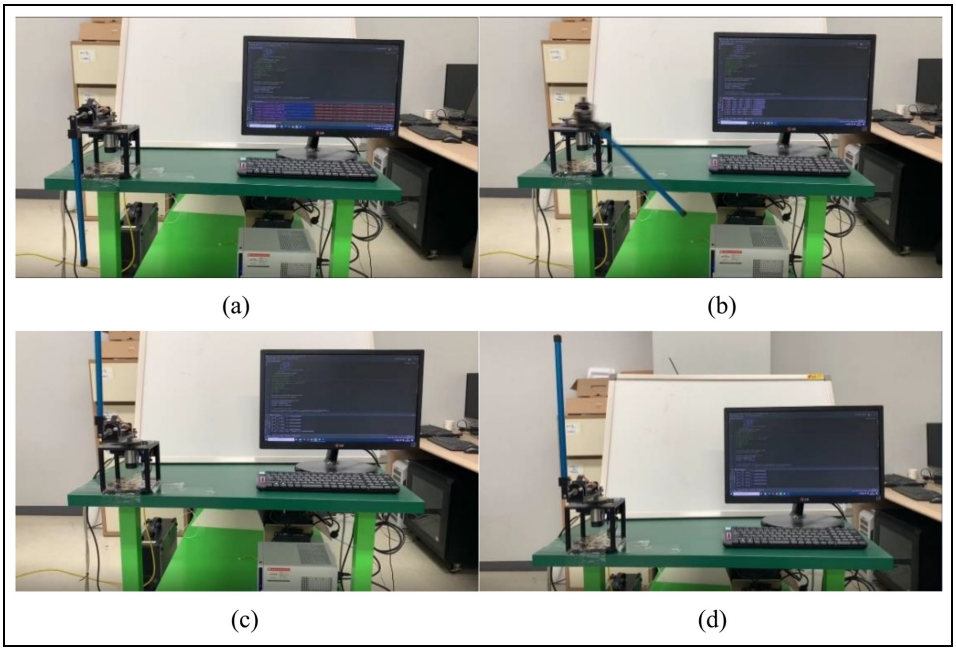

Figure 9. Snapshots of rotary inverted pendulum swing-up and balancing experiment using DDQN with prioritized experience replay: (a) at $t = 0$ s, the pendulum is in the downright position, (b) at $t = 2$ s the pendulum is on the swing-up process, (c) at $t = 5$ s the pendulum is in the upright position, and the rotary arm is in the central position, and (d) at $t = 11$ s, the pendulum is on the balancing process.

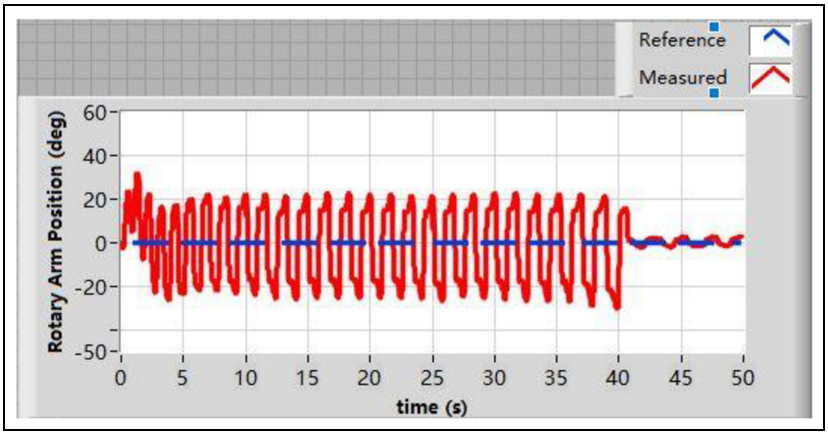

Figure 10. Rotary arm angle based on the classic control theory.

based on the classic control theory uses around 40s to swing up the pendulum, which is totally slower than the swing up process based on the DDQN with prioritized experience replay algorithm. We did the rotary inverted pendulum swing up and balance experiment 20 times, based on DDQN with prioritized experience replay and classic control theory, separately. If the pendulum is controlled based on DDQN with prioritized experience replay, it used 4.97 s (mean of 20 times value) to make the pendulum upright. However, if using classic control theory, it used 35.93 s (mean of 20 times value) to make the pendulum upright.

Figure 12 presents the reduction of overestimation performance comparing DQN and DDQN when doing the rotary inverted pendulum swing-up experience. Although there is still an overestimation, the reduction of overestimation performance using DDQN is better than the performance using DQN. When the pendulum is upright, $Q_{target}(s', argmaxQ_{eval}(s', a; \sigma), \sigma')$ is zero. The reward is calculated based on (4), and it is related to the rotary arm angle. When the pendulum is upright, the rotary arm rotates lightly, so the reward is close to zero, but not zero. Therefore, based on (5), when the inverted pendulum is upright, the Q value is close to zero, but not zero.

Figure 13 shows the training time performance comparing DDQN and DDQN with prioritized experience replay. We start from the time both methods obtain the

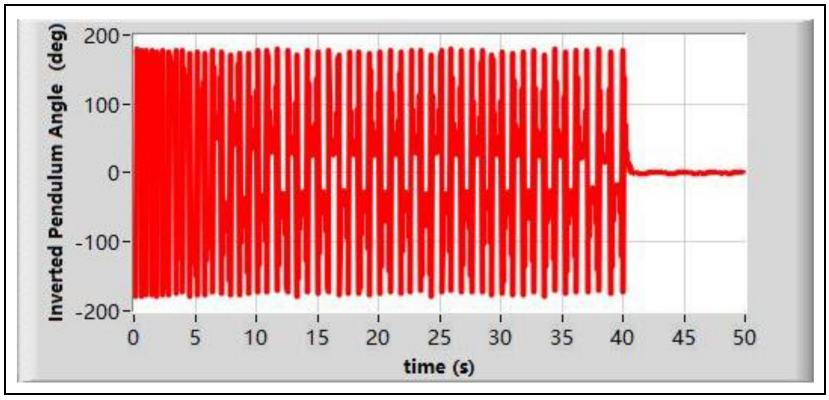

Figure 11. Inverted pendulum angle based on the classic control theory.

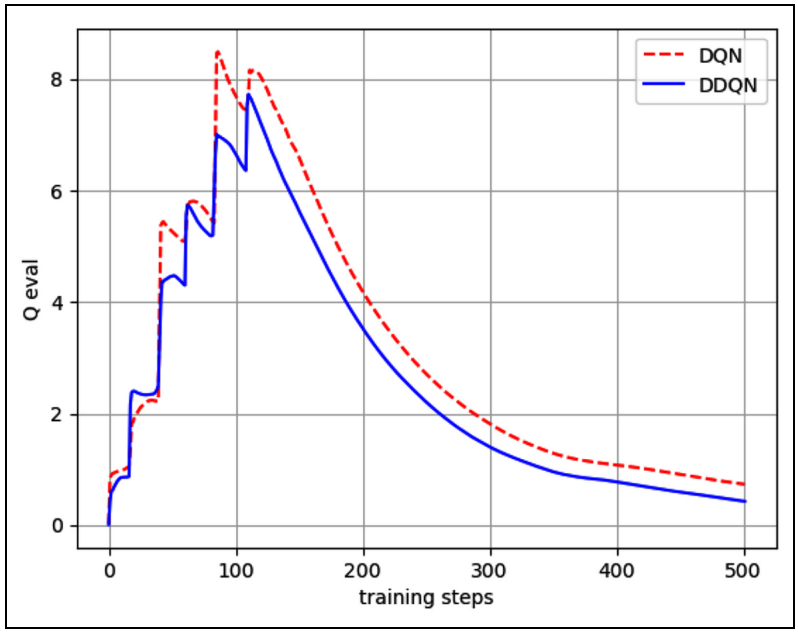

Figure 12. Reduction of overestimation performance comparing DQN and DDQN.

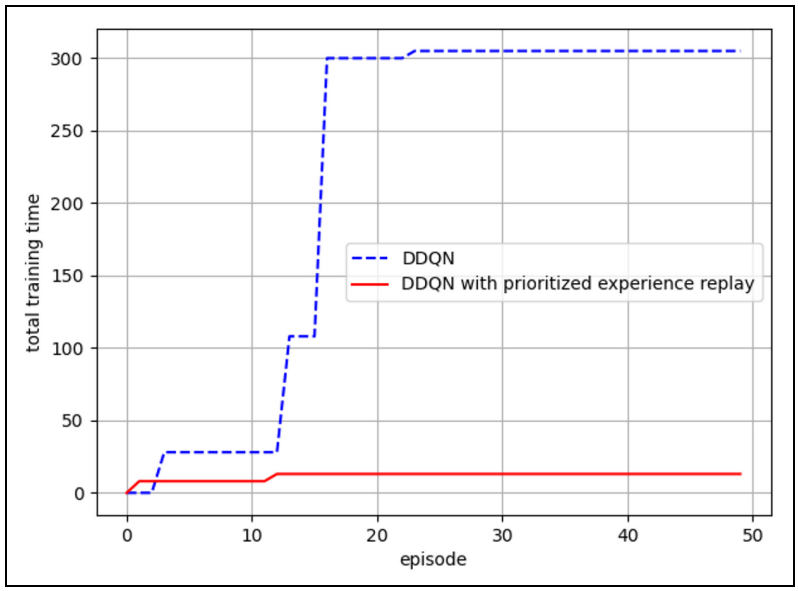

Figure 13. Training time performance comparing DDQN and DDQN with prioritized experience replay.

first reward. Every episode, the reward can be obtained using less steps with the prioritized experience replay algorithm. The rarely obtained reward can be used efficiently and be learned. Therefore, the prioritized experience replay algorithm helps to end each episode sooner and make the inverted pendulum upright.

Conclusion

In this paper, a real-time Hardware-in-the-loop (HIL) control system is proposed to swing up and balance a real rotary inverted pendulum by training and testing the deep reinforcement learning algorithm. The control system includes four parts: rotary inverted pendulum platform part, HIL interface software part, RL environment part, and agent part. The control system has the ability to respond to and solve the rapid and unpredictable changes of the inverted pendulum. By using the HIL approach, the use of computer simulation and actual experiments can be combined and show a great improvement in efficiency. Without a deep understanding of classical control engineering, the Double Deep Q-Network (DDQN) with prioritized experience replay algorithm is used to implement the rotary inverted pendulum swing-up and balancing the pendulum. For the real experiment, we define 21 actions to swing up and balance the rotary inverted pendulum and make the pendulum smoothly move. Finally, this paper shows the effective and efficient experiment results with comparisons of classic control theory and different reinforcement learning algorithms. Comparing DQN, the DDQN with prioritized experience replay algorithm removes the overestimate of Q value and decreases the training time. Using DDQN with prioritized experience replay algorithm, the pendulum can be faster swing up than using classic control algorithm.

Appendix

1. dropbox.com/s/h2q6pq2atzm0qsb/rotary%20inverted%20pendulum%20based%20on%20DDQN%20with%20prioritized%20experience%20replay.mp4?dl=0
2. dropbox.com/s/g8myo0a76iv7u9g/rotary%20inverted%20pendulum%20based%20on%20the%20classic%20control%20theory.mp4?dl=0

Declaration of conflicting interests

The author(s) declared no potential conflicts of interest with respect to the research, authorship, and/or publication of this article.

Funding

The author(s) received no financial support for the research, authorship, and/or publication of this article.

ORCID iD

Yanyan Dai https://orcid.org/0000-0003-2484-4585

Supplemental material

Supplemental material for this article is available online.

References

1. Nazari S. *Robust stability analysis for a class of extended linearization system*. Master Thesis, Northeastern University, Boston, 2011.
2. Zhang MH and Jing X. A bioinspired dynamics-based adaptive fuzzy SMC method for half-cr active suspension systems with input dead zones and saturations. *IEEE Trans Cybern*. Epub ahead of print 26 February 2020. DOI: 10.1109/TCYB.2020.2972322.
3. Chen H and Sun N. Nonlinear control of underactuated systems subject to both actuated and unactuated state constraints with experimental verification. *IEEE Trans Ind Electron* 2020; 67(9): 7702–7714.
4. Qian D, Ding H, Lee SG, et al. Suppression of chaotic oscillations in a complex biological system by disturbance observer-based derivative-integral terminal sliding mode. *IEEE/CAA J Autom Sin* 2020; 7(1): 126–135.
5. Aranda- Escolástica E, Guinaldo M, Santos M, et al. Control of a chain pendulum: a fuzzy logic approach. *Int J Comput Intell Syst* 2016; 9(2): 281–295.
6. Qian D, Tong S and Lee SG. Fuzzy-Logic-based control of payloads subjected to double-pendulum motion in overhead cranes. *Autom Constr* 2016; 65: 133–143.
7. Magdy M, Marhomy AE and Attia MA. Modeling of inverted pendulum system with gravitational search algorithm optimized controller. *Ain Shams Eng J* 2019; 10: 129–149.
8. Verma MK, Jha SK, Gaur P, et al. Artificial intelligence based on control of 3D inverted pendulum. In: *2012 IEEE 5th India International Conference on Power Electronics (IICPE)*, Delhi, India, 6–8 December 2012, pp.1–5. New York: IEEE.
9. Younis W and Abdelati M. Design and implementation of an experimental segway model. *AIP Conf Proc* 2009; 1107: 350–354.
10. Notué Kadjie A, Nwagoum Tuwa PR and Woafo P. An electromechanical pendulum robot arm in action: dynamics and control. *Shock Vib* 2017; 2017: Article ID: 3979384.
11. LeCun Y, Bengio Y and Hinton G.. Deep learning. *Nature* 2015; 521(7553): 436–444.
12. Wang ZY, Schaul T, Hessel M, et al. Dueling network architectures for deep reinforcement learning. In: *Proceedings of the 33rd International Conference on Machine Learning, PMLR*, 2016, vol. 48, New York: PMLR, pp.1995–2003.
13. Sutton RS and Barto AG. *Introduction to reinforcement learning*. Cambridge: MIT Press, 1998.
14. Watkins CJCH. *Learning from delayed rewards*. PhD thesis, University of Cambridge England, 1989.
15. Van Hasselt H, Guez A and Silver D. Deep reinforcement learning with double q-learning. arXiv preprint arXiv:1509.06461, 2015.

16. Thrun S and Schwartz A. Issues in using function approximation for reinforcement learning. In: *Proceedings of the fouth connectionist models summer school*, June 1993. Hillsdale, NJ: Lawrence Erlbaum Publisher.
17. van Hasselt H. Double Q-learning. *Adv Neural Inf Process Syst* 2010; 23: 2613–2621.
18. van Hasselt H, Guez A and Silver D. Deep reinforcement learning with double Q-learning. In: *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, February 2016, Palo Alto: AAAI Press, pp.2094–2100. Palo Alto: AAAI Press.
19. Schaul T, Quan J, Antonoglou I, et al. Prioritized experience replay. arXiv:1511.05952, 2016.
20. Polzounov K and Redden L. Blue river controls: a toolkit for reinforcement learning control systems on hardware. arXiv:2001.02254, 2020.
21. Kim JB, Kwon DH, Hong YG, et al. Deep Q-network based rotary inverted pendulum system and its monitoring on the EdgeX platform. In: *2019 International Conference on Artificial Intelligence in Information and Communication (ICAIIC)*, Okinawa, Japan, 11–13 February 2019, pp.34–39. New York: IEEE.
22. Quanser. *Rotary Inverted Pendulum*. Canada: Quanser Inc., 2012, https://www.quanser.com/products/rotary-inverted-pendulum/